

ML Quant Finance Project Ideas for Learning by Building

Executive summary

This report prioritizes technically deep, portfolio-worthy ML projects in quantitative finance, with learning value weighted more heavily than immediate practical utility. The strongest candidates are the ones that force you to solve several hard problems at once: point-in-time data engineering, nontrivial financial target design, realistic backtesting, and model choices that are constrained by market microstructure or no-arbitrage structure. Based on that lens, the highest-upside ideas are: a queue-aware limit-order-book foundation model, an arbitrage-aware option-surface forecasting system, a multimodal SEC-filings equity model, a point-in-time macro regime model, and a dynamic graph neural network for stock ranking. These areas are supported by strong primary-source literature and official data sources such as SEC EDGAR, FRED/ALFRED, Deribit, CME, Cboe, WRDS/CRSP, and LOBSTER, with notable open-source starting points such as ABIDES, PFHedge, FinBERT, Longformer, and the original RSR codebase. ¹

My recommended top project is **arbitrage-aware implied volatility surface forecasting on crypto options, with a deep-hedging extension**. It is unusually strong for learning-by-building because it combines derivative pricing, market microstructure, irregular-grid sequence modeling, no-arbitrage constraints, probabilistic forecasting, and realistic risk management, while still being feasible with publicly accessible data from Deribit and optional paid upgrades from Cboe. It is also distinctive enough to stand out in a portfolio or publication-style writeup, without requiring the institutional market data budget that a full equities limit-order-book project often needs. ²

A crucial assumption is explicit: **budget and compute were unspecified**. I therefore treat all compute estimates and timelines below as engineering estimates rather than hard requirements, and I separate free/public data paths from institutional or paid upgrades wherever it matters.

Assumptions and research basis

The target audience assumption is already fairly helpful: intermediate ML and finance knowledge. That means the best projects are not “intro trading bots,” but systems that teach hard lessons about causal data alignment, delayed information arrival, regime dependence, sparse targets, costs, capacity, and model risk. I also assume you want projects that can be built end-to-end by one person, even if some have an institutional-scale “stretch” version. Because up-to-date access conditions matter, dataset recommendations below rely on official or primary sources where possible: SEC EDGAR for filings and XBRL, FRED/ALFRED for macro series and vintages, Deribit and Cboe for options data, CME for derivatives history, WRDS/CRSP for academic equity data, and LOBSTER for full-depth Nasdaq order-book data. ³

The model suggestions are likewise anchored in primary or highly influential sources. For time series and macro forecasting, the strongest modern baselines include Temporal Fusion Transformer and PatchTST; for

regime-switching dynamics, deep switching state-space models are directly relevant. For document-heavy fundamental modeling, FinBERT and Longformer are canonical starting points. For market microstructure, DeepLOB, transformer-based LOB models, LiT, fill-probability survival models, and the recent TradeFM foundation-model direction are the right conceptual ladder. For graph-based equity modeling, the strongest classical anchor is Temporal Relational Stock Ranking. For derivative hedging under frictions, the deep-hedging framework and PFHedge are highly useful reference implementations. ⁴

The ranking tables below are **my synthesis**, not mechanically copied from any single source. The scores reflect the user's stated priority order: technical depth and learning value first, usefulness second.

Comparative ranking and selection

The first table compares the projects across the attributes you explicitly asked for.

Project	Learning value	Feasibility	Data difficulty	Compute	Novelty	Impact	Best fit
Queue-aware LOB foundation model	10/10	4/10	Very high	High	10/10	9/10	Maximum microstructure depth
Arbitrage-aware options surface forecasting	9.5/10	7/10	Medium-high	Medium-high	9.5/10	9/10	Best technical balance
Multimodal SEC-filings equity model	9/10	8/10	High	Medium-high	8.5/10	9/10	Best broad portfolio project
Dynamic graph stock ranking	9/10	6/10	High	Medium-high	9/10	8.5/10	Best relational ML project
Point-in-time macro regime futures model	8.5/10	8/10	Medium	Medium	8/10	8/10	Best signal-engineering project

The next table gives a more explicit prioritization by criterion.

Priority axis	Rank order
Learning value	LOB foundation model → Options surface forecasting → SEC multimodal equity model → Dynamic graph stock ranking → Macro regime futures model
Feasibility	Macro regime futures model → SEC multimodal equity model → Options surface forecasting → Dynamic graph stock ranking → LOB foundation model
Portfolio/publication impact	SEC multimodal equity model → Options surface forecasting → LOB foundation model → Dynamic graph stock ranking → Macro regime futures model
Recommended overall for your brief	Options surface forecasting

The reason I recommend the **options surface project** over the others is not that it is “easier”; it is that it gives you an unusually rich combination of topics that are genuinely hard in quant finance: surface construction from sparse noisy quotes, no-arbitrage constraints, term-structure dynamics, non-Euclidean grid design over strike and maturity, realistic hedging evaluation, and risk-factor interpretation. It is also more feasible than the order-book project because Deribit offers public API access to options and historical retrieval endpoints, whereas professional-grade full-depth equities data often pushes you toward LOBSTER or exchange data products. ⁵

Project options

Queue-aware LOB foundation model for alpha, fill probability, and execution

This project starts from the modern microstructure literature: DeepLOB showed that CNN-plus-sequence architectures can extract robust predictive structure from book states; transformer-based LOB models and later transformer-only designs pushed that direction further; a related survival-modeling line estimates fill-time distributions for passive orders; ABIDES and ABIDES-Gym provide realistic multi-agent simulation for execution; and the newer TradeFM paper points toward foundation-model-style pretraining over market event streams. Public benchmark access exists through FI-2010, while LOBSTER provides much richer Nasdaq order-book data for academic and commercial users. ⁶

Aspect	Recommendation
Concise description	Pretrain a sequence model on order-book events or snapshots, then fine-tune it jointly for short-horizon mid-price movement, fill-time prediction, and execution-policy support.
Why technically deep	You must handle event-time sampling, label leakage risk, queue dynamics, survival targets, simulator integration, and extremely noisy low-latency signals.

Aspect	Recommendation
Datasets	Free MVP: FI-2010 benchmark. Better public path: self-collected crypto L2/L3 data. Institutional path: LOBSTER or exchange-grade depth data. Simulation path: ABIDES/ABIDES-Gym.
Suggested models	Baselines: logistic regression on OFI features, DeepLOB, causal CNN/LSTM. Main model: causal Transformer with masked-event or next-event pretraining and a multitask head for classification + survival.
Feature engineering and signal design	Raw top-of-book and top-10 levels, order-flow imbalance, queue imbalance, spread, relative depth by level, recent cancel/market-order intensity, event-time realized volatility, and instrument-normalized relative price coordinates.
Evaluation and backtesting	Predictive: balanced accuracy, F1, AUROC, Brier score, reliability diagrams, concordance for fill models. Trading: event-driven walk-forward backtest, fee model, maker-taker treatment, queue-position simulation, slippage by aggressive participation, and latency-aware decision delays.
Risk controls	Inventory caps, max order size as share of visible depth, max participation rate, kill switch after spread widening or volatility spikes, and aggressive-to-passive switching rules when fill odds collapse.
Compute and tooling	Strong MVP on one 24 GB GPU; institutional-scale pretraining benefits from multi-GPU. Tooling: PyTorch, Lightning, Polars/Parquet, ABIDES, Hydra, Weights & Biases.
Estimated timeline and difficulty	10–14 weeks. Difficulty: very high .
Extensions	Cross-asset transfer learning, synthetic-event generation à la TradeFM, policy-learning in ABIDES, causal stress testing, or a paper-style ablation study on queues and horizons.

Arbitrage-aware implied volatility surface forecasting on crypto options

This is the recommended top project. Deribit’s official API supports options, futures, and historical access patterns, which makes it one of the few venues where a serious options ML project can be prototyped without immediately buying institutional data. On the modeling side, the literature on arbitrage-free surface prediction and generation gives you a clear research backbone: two-step prediction under static no-arbitrage constraints, VAE-plus-SDE approaches for arbitrage-free surfaces, and newer deep-learning formulations that enforce no-arbitrage directly. If you want a realistic downstream task, deep hedging and PFHedge are natural extensions. Paid upgrades from Cboe DataShop can move the project closer to U.S. listed options later. ⁷

Aspect	Recommendation
Concise description	Forecast the next-day or intraday implied-volatility surface across strike and maturity, with a decoder or penalty scheme that suppresses butterfly and calendar arbitrage; then evaluate not just forecast error, but hedging value and tradability.
Why technically deep	Surface data are sparse, noisy, irregular, and heavily structured by finance constraints; success requires combining representation learning with option theory rather than treating the problem as ordinary image forecasting.
Datasets	Free/public path: Deribit options chains, historical trades, underlying spot/futures, and realized-volatility endpoints. Paid upgrade: Cboe historical options and DataShop products.
Suggested models	Baselines: SVI/SSVI or SABR factor models plus AR terms on latent parameters. Main model: latent-factor encoder-decoder over total variance, with Transformer, TFT, or state-space temporal dynamics and explicit arbitrage penalties or constrained output layers. Optional extension: neural process or VAE-based surface reconstruction.
Feature engineering and signal design	Forward moneyness, log-moneyness, time to expiry, delta bucket, vega, bid-ask width, quote staleness, open interest, surface PCA factors, funding basis, realized-volatility term structure, vol-of-vol proxies, skew slope, term-structure curvature, and event flags around large moves.
Evaluation and backtesting	Forecasting: vega-weighted RMSE/MAE, total-variance error, calibration by tenor and moneyness bucket, arbitrage-violation counts and magnitudes. Trading and hedging: out-of-sample delta- or vega-neutral strategy PnL, hedging error, turnover, transaction costs, slippage from crossing the spread, and stress tests under jumpy underlying moves. Use nested walk-forward windows by calendar date.
Risk controls	Greek limits, exposure caps by expiry and moneyness wing, stale-quote filters, minimum-liquidity thresholds, max overnight gamma, scenario shocks to skew and term structure, and hard rejection of surfaces with excessive arbitrage penalty.
Compute and tooling	A strong version fits on one midrange modern GPU; an excellent version benefits from a 24–48 GB GPU. Tooling: PyTorch, JAX or QuantLib for pricing utilities, PFHedge for hedging experiments, DuckDB/Polars for chain processing.
Estimated timeline and difficulty	8–12 weeks. Difficulty: high .
Extensions	Deep hedging under costs, stochastic local-volatility calibration, generative surface simulation, cross-venue transfer, or a publication-style paper on “ML under no-arbitrage constraints.”

Multimodal SEC-filings, fundamentals, and macro model for cross-sectional equity returns

This is the most versatile portfolio project. The SEC’s EDGAR APIs provide submissions and XBRL filing data in JSON without authentication; FRED and ALFRED provide macro and vintage data; and canonical empirical

asset-pricing work by Gu, Kelly, and Xiu gives you the right evaluation language for expected-return prediction. On the NLP side, FinBERT is a standard finance-domain encoder, while Longformer is useful when you actually want to process full MD&A or Risk Factors sections rather than headlines. For prices and fundamentals, the institutional route is CRSP/Compustat via WRDS; a practical alternative is Sharadar via Nasdaq Data Link, with a lighter API-style path via Massive stock aggregates. ⁸

Aspect	Recommendation
Concise description	Build a point-in-time cross-sectional return model that fuses structured accounting data, macro context, and long-document filing text, then test it as a ranked stock-selection system.
Why technically deep	The difficult parts are not just modeling; they are timestamp hygiene, issue-date alignment, revisions, delistings, survivorship bias, and long-document compression under finance-domain vocabulary.
Datasets	Free/public core: SEC EDGAR submissions/XBRL and FRED/ALFRED. Best academic path: WRDS CRSP/Compustat. Practical paid alternative: Nasdaq Data Link Sharadar. Lightweight price option: Massive stock aggregates.
Suggested models	Structured tower: boosted trees or TabTransformer/MLP. Text tower: FinBERT or Longformer with section-wise pooling over MD&A, risk factors, and earnings comments. Fusion: gated late fusion or mixture-of-experts trained with cross-sectional ranking loss.
Feature engineering and signal design	Classic characteristics such as momentum, reversal, volatility, liquidity, investment, profitability, accruals, issuance, and size; textual sentiment, uncertainty, similarity-to-prior-filing, topic drift, risk-factor novelty, and management-tone dispersion; macro regime features from ALFRED vintages.
Evaluation and backtesting	Cross-sectional IC, rank IC, top-decile minus bottom-decile, hit rate, turnover, sector-neutral and beta-neutral variants, drawdown, capacity proxies, and transaction-cost-adjusted walk-forward backtests. Use strictly point-in-time snapshots and embargo around filing dates where needed.
Risk controls	Beta, sector, industry, size, and residual-volatility neutrality; single-name caps; ADV caps; earnings blackout windows if desired; and turnover penalties during optimization.
Compute and tooling	CPU is enough for the structured-only MVP; one GPU is highly helpful once Longformer enters the stack. Tooling: PyTorch/Transformers, Polars, DuckDB, Dask/Ray if you scale across many filings.
Estimated timeline and difficulty	10–14 weeks. Difficulty: high .
Extensions	Event studies on 8-Ks, retrieval-augmented filing summarization, factor attribution, international filings, or a reproducible “point-in-time multimodal asset pricing” repo.

Dynamic graph neural network for relational stock ranking

The graph project is highly impressive when executed well because it moves beyond “each stock is an IID time series.” Temporal Relational Stock Ranking is the classical anchor here: it explicitly models relations among stocks and optimizes for ranking rather than price regression. Later work explored GNNs in rolling-window market settings, and the original authors also released code and open relation files such as industry and wiki relations. A stronger modern build, though, uses SEC filing text and metadata to infer supplier, customer, competitor, and thematic links dynamically instead of relying only on static sector adjacency. ⁹

Aspect	Recommendation
Concise description	Build a time-varying company graph and use a temporal graph model to rank stocks by expected cross-sectional return or residual return.
Why technically deep	You must solve graph construction, edge freshness, relation uncertainty, cross-sectional ranking, and temporal dependence simultaneously.
Datasets	Graph sources: author-provided industry/wiki relations from the RSR repo for an open benchmark, plus SEC EDGAR filings to mine customer/supplier/competitor edges. Market data: WRDS/CRSP, Sharadar, or a lighter API source.
Suggested models	Baselines: sector dummy model, similarity graph with linear propagation, RSR-style temporal graph convolution. Main model: temporal graph attention network with node encoders for returns/fundamentals/text and edge features for relation type, confidence, and recency.
Feature engineering and signal design	Node features from returns, characteristics, and filing embeddings; edge features from co-mentions, supplier intensity, sector links, knowledge-graph distance, and rolling correlation residuals. Targets should be ranked future residual returns, not raw prices.
Evaluation and backtesting	Rank IC, top-k spread returns, graph-edge ablations, stability across rebalancing frequencies, turnover and cost-adjusted long-short performance, and tests of whether dynamic edges beat static sector edges. Walk-forward retraining is mandatory.
Risk controls	Sector and cluster-neutral portfolio construction, concentration caps by graph community, edge-confidence thresholds, and turnover regularization because graph drift can induce excessive trading.
Compute and tooling	One GPU is sufficient for most versions; data engineering is the larger bottleneck. Tooling: PyTorch Geometric or DGL, DuckDB/Polars, SEC parsers, and perhaps Neo4j for graph inspection.
Estimated timeline and difficulty	10–14 weeks. Difficulty: high .
Extensions	Relation extraction from filing sections using LLMs, graph contrastive learning, regime-specific graphs, or publication-style analysis of relation decay.

Point-in-time macro regime model for futures allocation

This is the cleanest choice if you want a project that teaches backtesting discipline and signal engineering as much as deep learning. ALFRED is particularly valuable because it records historical vintages and release dates, which helps you avoid the common mistake of training on revised data that were not available at the decision time. Combined with FRED and exchange-grade historical futures data from CME DataMine, you can construct a serious macro-trading research stack. On the modeling side, deep switching state-space models are specifically aimed at nonlinear regime-switching time series, while TFT and PatchTST provide strong multivariate forecasting baselines. ¹⁰

Aspect	Recommendation
Concise description	Forecast macro-sensitive futures returns using a point-in-time feature set built from unrevised macro releases, release surprises, and market term structures, with an explicit latent-regime model.
Why technically deep	The key difficulty is data realism: vintage alignment, release timing, mixed frequencies, asynchronous assets, and the fact that macro signals often work only conditionally by regime.
Datasets	Official macro core: ALFRED/FRED vintages and release metadata. Institutional market path: CME historical futures via DataMine. Lower-budget proxy path: highly liquid macro ETFs or rates products via a stock/ETF API, with the caveat that this is a weaker substitute.
Suggested models	Baselines: linear surprise models, HMM, gradient boosting. Main model: deep switching state-space model or TFT/PatchTST with regime-conditioned experts.
Feature engineering and signal design	Release surprises, revisions, yield-curve slope and curvature, carry, basis, trend, realized-volatility regimes, inflation-growth composites, policy-rate path repricing, and cross-asset confirmation signals.
Evaluation and backtesting	Directional accuracy, NLL or CRPS for probabilistic targets, walk-forward futures portfolio PnL, volatility-targeted Sharpe, drawdown, and performance conditioned on expansions, recessions, and policy-turning points. Costs and slippage should scale with contract liquidity and bid-ask estimates.
Risk controls	Volatility targeting, max gross leverage, contract concentration caps, crisis throttles, and explicit handling of roll windows and roll costs.
Compute and tooling	Mostly moderate; a CPU-heavy MVP is viable and one GPU is enough for deep sequence models. Tooling: pandas/Polars, PyTorch Forecasting or NeuralForecast, and your own release-calendar alignment utilities.
Estimated timeline and difficulty	6–10 weeks. Difficulty: medium-high .
Extensions	Nowcasting macro surprises, macro-factor attribution, global macro replication, or a paper-style study of revised versus unrevised data leakage.

Recommended top project

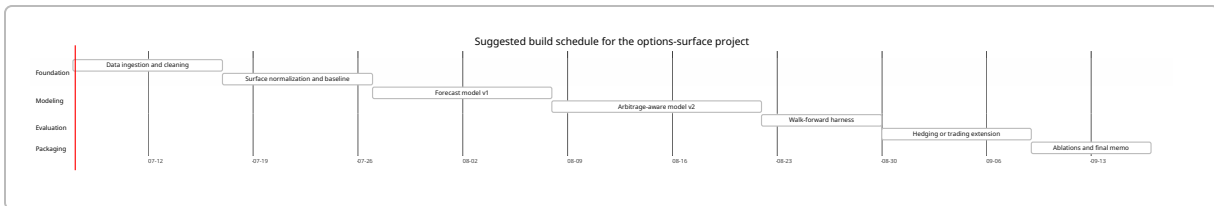
The project I would actually build first, given your brief, is **arbitrage-aware implied volatility surface forecasting on Deribit options with a downstream hedging evaluation**. It is the rare project that is simultaneously technically rich, quant-legible, and feasible without a full institutional data stack. It forces you to learn the right abstractions: use total implied variance rather than raw IV when appropriate, normalize by forward moneyness, separate static cross-sectional structure from temporal dynamics, and treat no-arbitrage as a modeling constraint rather than a post hoc check. It also produces deliverables that are easier to explain than a black-box alpha strategy: surface heatmaps, arbitrage maps, forecast vs realized slices, Greek exposures, and hedging-cost reductions. ¹¹

A good version has three layers. The first layer is **surface construction**: ingest chains, clean and filter quotes, convert to forward-moneyness coordinates, and fit or interpolate a baseline surface. The second layer is **forecasting**: build a model that predicts the next surface or latent surface factors while minimizing vega-weighted forecast error and penalizing arbitrage. The third layer is **financial evaluation**: convert the forecast to an actionable task such as delta/vega hedging, relative-value trades, or surface reconstruction under costs. That three-layer structure is what makes the project feel like quant finance rather than generic deep learning. ¹²

Detailed development plan

Milestone	Deliverables	Notes
Data acquisition and schema	Raw Deribit chain store, instrument metadata, underlying time series, liquidity filters, and daily/intraday snapshots	Start with BTC options only; ETH is a clean extension.
Surface normalization and baselines	Forward-price construction, moneyness buckets, total-variance targets, SVI/SSVI or SABR baseline fit, arbitrage checker	This stage forces the finance conventions into the pipeline.
Forecast model v1	Latent-factor temporal model and vega-weighted loss	A lightweight MLP, GRU, or TFT baseline is enough for the first serious result.
Arbitrage-aware model v2	Penalty terms or constrained decoder, bucketed evaluation dashboard, tenor/moneyness error maps	This is the real technical center of the project.
Walk-forward research harness	Rolling train/validation/test splits, hyperparameter logs, model registry, cost-aware backtester	Keep all evaluation out-of-sample by calendar date.
Hedging or trading extension	Delta/vega-neutral hedging simulation with PFHedge-style architecture or a simpler execution harness	Do not skip this if you want the project to feel like quant research rather than interpolation.

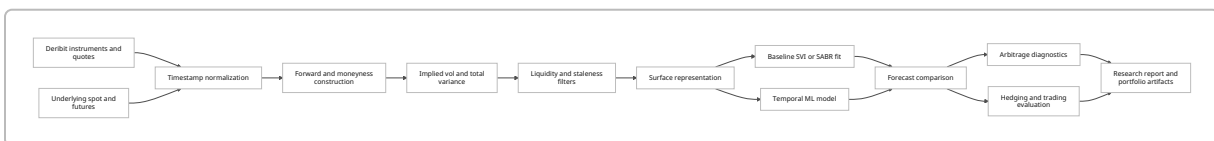
Milestone	Deliverables	Notes
Writeup and packaging	Reproducible repo, experiment cards, charts, ablations, and a research-style memo	Include failures, not just best results.



Data pipeline outline

The pipeline should be explicit and point-in-time clean.

Stage	What happens	Failure modes to watch
Ingest	Pull instrument definitions, option quotes/trades, underlying spot/futures, historical volatility, and order-book snapshots if needed	Missing instruments, duplicate snapshots, inconsistent timestamps
Normalize	Build forward price, map quotes to log-forward moneyness and expiry, compute implied vols and total variance	Bad forwards, American/European mismatch assumptions, stale mids
Filter	Drop wide-spread or low-liquidity quotes, cap stale marks, exclude pathological wings	Surface artifacts from illiquid wings
Represent	Store irregular quote sets and optionally resample to a dense strike-tenor grid	Over-smoothing sparse data
Model	Fit baseline surface, then train temporal model with arbitrage penalty	Data leakage across walk-forward windows
Evaluate	Forecast error, arbitrage diagnostics, hedge PnL, cost-adjusted metrics	Fantastic interpolation but no financial value
Publish	Save model cards, plots, notebooks, and reproducible experiment configs	Non-reproducible results



Core components

A clean first code path is better than a “fancy” one. The first snippet below sketches the data ingestion and feature construction logic.

```
# Pseudocode: Deribit surface ingestion
from dataclasses import dataclass
import pandas as pd
import numpy as np

@dataclass
class Quote:
    ts: pd.Timestamp
    instrument: str
    bid_iv: float
    ask_iv: float
    bid_price: float
    ask_price: float
    mark_iv: float
    underlying_price: float
    open_interest: float

def build_surface_frame(quotes: pd.DataFrame, instruments: pd.DataFrame) ->
pd.DataFrame:
    df = quotes.merge(instruments, on="instrument", how="left")

    # Forward proxy: use futures if available; otherwise a documented
    approximation.
    df["tau_years"] = (df["expiry"] - df["ts"]).dt.total_seconds() / (365.25 *
24 * 3600)
    df = df[df["tau_years"] > 0]

    # Mid IV with staleness and spread checks
    df["mid_iv"] = np.where(
        (df["bid_iv"] > 0) & (df["ask_iv"] > 0),
        0.5 * (df["bid_iv"] + df["ask_iv"]),
        df["mark_iv"]
    )

    # Prefer forward moneyness if forward is available
    df["forward"] = df["underlying_price"] # replace with futures-implied
    forward when available
    df["log_moneyness"] = np.log(df["strike"] / df["forward"])
    df["total_variance"] = (df["mid_iv"] ** 2) * df["tau_years"]

    # Liquidity and hygiene filters
    valid = (
```

```

    df["mid_iv"].between(0.01, 5.0)
    & (df["open_interest"].fillna(0) >= 0)
    & df["log_moneyness"].replace([np.inf, -np.inf], np.nan).notna()
)
return df.loc[valid].copy()

```

The central modeling idea is to predict **total variance** and penalize violations of static no-arbitrage. The penalty can be approximate at first; the important part is to make it part of training.

```

# Pseudocode: loss with simple arbitrage penalties
import torch
import torch.nn.functional as F

def butterfly_penalty(total_var_grid: torch.Tensor) -> torch.Tensor:
    """
    total_var_grid shape: [batch, tenor, strike]
    A simple convexity proxy in strike dimension.
    """
    second_diff = (
        total_var_grid[:, :, 2:]
        - 2.0 * total_var_grid[:, :, 1:-1]
        + total_var_grid[:, :, :-2]
    )
    return F.relu(-second_diff).mean()

def calendar_penalty(total_var_grid: torch.Tensor) -> torch.Tensor:
    """
    Monotonicity proxy in tenor dimension.
    """
    tenor_diff = total_var_grid[:, 1:, :] - total_var_grid[:, :-1, :]
    return F.relu(-tenor_diff).mean()

def vega_weighted_mse(pred_iv, true_iv, vegas, eps=1e-6):
    w = vegas / (vegas.mean() + eps)
    return (w * (pred_iv - true_iv) ** 2).mean()

def total_loss(pred_iv, true_iv, vegas, tau, lam_bfly=10.0, lam_cal=10.0):
    pred_w = (pred_iv ** 2) * tau
    base = vega_weighted_mse(pred_iv, true_iv, vegas)
    return base + lam_bfly * butterfly_penalty(pred_w) + lam_cal *
    calendar_penalty(pred_w)

```

The research harness should be fully walk-forward. For a serious project, resist the temptation to random-split dates.

```

# Pseudocode: rolling walk-forward evaluation
def walk_forward_splits(dates, train_days=180, val_days=30, test_days=30,
step_days=30):
    dates = sorted(pd.to_datetime(pd.Series(dates).unique()))
    i = 0
    while True:
        train_start = i
        train_end = train_start + train_days
        val_end = train_end + val_days
        test_end = val_end + test_days
        if test_end > len(dates):
            break
        yield {
            "train": dates[train_start:train_end],
            "val": dates[train_end:val_end],
            "test": dates[val_end:test_end],
        }
        i += step_days

for split in walk_forward_splits(surface_dates):
    train_df = surface_df[surface_df["date"].isin(split["train"])]
    val_df = surface_df[surface_df["date"].isin(split["val"])]
    test_df = surface_df[surface_df["date"].isin(split["test"])]

    model = build_model()
    fit(model, train_df, val_df)
    preds = predict(model, test_df)
    evaluate_surface_errors(preds, test_df)
    evaluate_hedging_pnl(preds, test_df, costs=True, slippage=True)

```

Backtesting methodology for the top project

The backtest should not stop at “surface RMSE.” Use a nested walk-forward design with rolling or expanding windows. At each fold, fit on historical dates only, select hyperparameters on a chronologically later validation segment, and report results only on the final unseen test segment. For an intraday variant, align all targets to the last quote set actually available at decision time and include a latency buffer. The minimum financial evaluation should include vega-weighted forecast error, arbitrage diagnostics, hedging error for a simple option inventory, transaction costs from crossing the spread, and scenario stress tests for large underlying jumps. This is exactly where deep hedging becomes useful as an extension rather than a separate project: it tells you whether the forecast model is improving a real control problem under frictions.

13

For slippage and cost assumptions, start conservative. Crossing one half-spread is the minimum honest assumption for aggressive re-hedging, and wider effective cost assumptions should be used in stressed regimes or wing strikes. Evaluate under at least three cost regimes: optimistic, base, and stressed. Also

report exposure stability by net delta, vega, and gamma. If the model only works by taking unstable wing exposure or excessive turnover, it is not yet a good quant project even if its headline RMSE is attractive.

Implementation notes and portfolio positioning

For a portfolio or publication-style presentation, the most convincing artifact is not a glossy dashboard; it is a **research narrative**. That means documenting the pipeline, ablations, and failures. Show the baseline SVI/SABR or factor model first, then show what the ML model improves, then show what the arbitrage constraints preserve, then show whether hedging or relative-value performance survives costs. If you choose the SEC multimodal project instead, the same logic applies: start with a structured model only, then add text, then add point-in-time macro, then show what each piece actually contributes. That style mirrors how good empirical finance papers argue for incremental value. ¹⁴

If your goal is maximum prestige per unit effort, the recommended ordering is simple. Build the **options surface project** first. If you later want a second project, do the **SEC multimodal asset-pricing project**, because it broadens your profile from derivatives and structured prediction into point-in-time fundamentals and financial NLP. If you want the purest “hard mode” challenge after that, tackle the **LOB project** only once you have a clean story around data realism and event-driven evaluation. The macro and graph projects are excellent third or fourth projects because they teach different research reflexes rather than repeating the same ones. ¹⁵

The final practical note is the same across all projects: keep a strict separation between **predictive metrics**, **economic metrics**, and **risk metrics**. In quant finance, good projects fail because they optimize the first while ignoring the second and third. The best projects above are impressive precisely because they force you to respect all three.

¹ ⁶ DeepLOB: Deep Convolutional Neural Networks for Limit Order Books

https://arxiv.org/abs/1808.03668?utm_source=chatgpt.com

² ⁵ ⁷ ¹¹ ¹⁵ Deribit API Documentation - Cryptocurrency Derivatives ...

https://docs.deribit.com/?utm_source=chatgpt.com

³ ⁸ EDGAR Application Programming Interfaces (APIs)

https://www.sec.gov/search-filings/edgar-application-programming-interfaces?utm_source=chatgpt.com

⁴ Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting

https://arxiv.org/abs/1912.09363?utm_source=chatgpt.com

⁹ Temporal Relational Ranking for Stock Prediction

https://arxiv.org/abs/1809.09441?utm_source=chatgpt.com

¹⁰ ALFRED | St. Louis Fed: Archival FRED

https://alfred.stlouisfed.org/?utm_source=chatgpt.com

¹² A Two-Step Framework for Arbitrage-Free Prediction of the ...

https://arxiv.org/abs/2106.07177?utm_source=chatgpt.com

¹³ [1802.03042] Deep Hedging

https://arxiv.org/abs/1802.03042?utm_source=chatgpt.com

¹⁴ Empirical Asset Pricing via Machine Learning - Oxford Academic
https://academic.oup.com/rfs/article/33/5/2223/5758276?utm_source=chatgpt.com